ECEN/CSEN 127
Homework #4
Dylan Thornburg

You are going to build a FIFO buffer circuit based on a register file.

1) Implement a 12-register register file. Each register needs to be 8 bits wide. There needs to be one read port and one write port.

The FIFO must be capable of holding 12 data elements at a time before becoming full.

```
module eightBitRegFile (
        input clk,
        input reset, // not required
        input write_enable,
        input [3:0] write_addr,
        input [7:0] write_data,
        input [4:0] read_addr,
        output [7:0] read_data
);
        wire [11:0] en;
        wire [7:0] Q [11:0];
        //despite what you may think, a case statement is actually more lines of code
        assign en[0] = write_enable && (write_addr == 4'd0);
        assign en[1] = write_enable && (write_addr == 4'd1);
        assign en[2] = write_enable && (write_addr == 4'd2);
        assign en[3] = write_enable && (write_addr == 4'd3);
```

```
        assign en[4] = write_enable && (write_addr == 4'd4);
        assign en[5] = write_enable && (write_addr == 4'd5);
        assign en[6] = write_enable && (write_addr == 4'd6);
        assign en[7] = write_enable && (write_addr == 4'd7);
        assign en[8] = write_enable && (write_addr == 4'd8);
        assign en[9] = write_enable && (write_addr == 4'd9);
        assign en[10] = write_enable && (write_addr == 4'd10);
        assign en[11] = write_enable && (write_addr == 4'd11);
        // added reset to make analyzing easier
        eightBitReg zero(write_data, clk, en[0], reset, Q[0]);
        eightBitReg one(write_data, clk, en[1], reset, Q[1]);
        eightBitReg two(write_data, clk, en[2], reset, Q[2]);
        eightBitReg three(write_data, clk, en[3], reset, Q[3]);
        eightBitReg four(write_data, clk, en[4], reset, Q[4]);
        eightBitReg five(write_data, clk, en[5], reset, Q[5]);
        eightBitReg six(write_data, clk, en[6], reset, Q[6]);
        eightBitReg seven(write_data, clk, en[7], reset, Q[7]);
        eightBitReg eight(write_data, clk, en[8], reset, Q[8]);
        eightBitReg nine(write_data, clk, en[9], reset, Q[9]);
        eightBitReg ten(write_data, clk, en[10], reset, Q[10]);
        eightBitReg eleven(write_data, clk, en[11], reset, Q[11]);
        //Bypass. If read addr == write addr and write is asserted, write data is read
        assign read_data = write_enable && (write_addr == read_addr) ? write_data :
Q[read_addr];
endmodule
```

2) Use this register file as part of the datapath of a FIFO buffer. Add a control path as well. The inputs are:

Din – 8 bits
Push – adds Din to the end of the FIFO buffer on the next rising clock edge

Pop – removes the first element from the FIFO buffer on the next rising clock edge
Clk

The outputs are:

Dout – the value of the first FIFO element
Full – true if the FIFO is full
Empty – true if the FIFO is empty

```
module fifo(
        input clk,
        input reset, //my regfile from lab has reset and I like it
        input push,
        input pop,
        input [7:0] din,
        output [7:0] dout,
        output full,
        output empty
);
```

3) Implement the datapath and control path

```
module fifo(
        input clk,
        input reset, //my regfile from lab has reset and I like it
        input push,
        input pop,
```

```
	input [7:0] din,
	output [7:0] dout,
	output full,
	output empty
);

	reg [3:0] write_addr, read_addr, count;
	wire [7:0] read_data;
	wire [3:0] count_next = count + (push && (count < 4'b1100)) - (pop && (count > 0));
	assign full = (count == 4'b1100);
	assign empty = (count == 0);
	assign dout = empty ? 1'bx : read_data;

	eightBitRegFile one (clk, reset, (push && (count < 4'b1100)), write_addr, din, read_addr,
read_data);

	assign dout = read_data;


	always @(posedge clk) begin
		if (reset) begin
			write_addr <= 0;
			read_addr <= 0;
			count <= 0;
		end
		else begin
			if (push && (count < 4'b1100)) write_addr <= (write_addr == 4'b1011) ?
0: write_addr + 1;
			if (pop && (count > 0)) read_addr <= (read_addr == 4'b1011) ? 0:
read_addr + 1;
			count <= count_next;
		end
	end
endmodule
```

4) Create a testbench and test it.

```
module testbench();

reg clk, reset, push, pop;
reg [7:0] din;
wire [7:0] dout;
wire full, empty;

fifo dut(clk, reset, push, pop, din, dout, full, empty);

initial begin
$display("Start of the FIFO Test");
$monitor("push=%d, pop=%d, din=%d, dout=%d, full=%d, empty=%d, reset=%d, clk=%d, @
%d", push, pop, din, dout, full, empty, reset, clk, $time);

$display("\n");

$display("starting reset (x instead of 0 on dout to symbolize empty. 0 on dout doesnt necessarily
mean nothing. I could be storing 0 in the fifo, but not x.)");
clk=0; reset=1; push=0; pop=0; din=0;
#10
clk=1; reset=1; push=0; pop=0; din=0;
#10
$display("\n");

$display("push 10");
clk=0; reset=0; push=1; pop=0; din=8'd10;
#10
clk=1; reset=0; push=1; pop=0; din=8'd10;
```

```
#10
$display("\n");

$display("push 20");
clk=0; reset=0; push=1; pop=0; din=8'd20;
#10
clk=1; reset=0; push=1; pop=0; din=8'd20;
#10
$display("\n");

$display("push data until full");
repeat (10) begin
        #10
        clk=0; push=1; pop=0; din=din+8'd10; //add up to 120 for full buffer
        #10
        clk=1; push=1; pop=0; din=din;
end
#10 // weird delay logic for the repeats. Cant have #num and then end. Ridiculous
$display("\n");

$display("try pushing while full");
clk=0; push=1; pop=0; din=din+8'd10;
#10
clk=1; push=1; pop=0; din=din;
#10
$display("\n");

$display("pop first element");
clk=0; push=0; pop=1; din=0;
#10
clk=1; push=0; pop=1; din=0;
#10
$display("\n");
```

```
$display("pop elements until empty");
repeat(11) begin
        #10
        clk=0; push=0; pop=1; din=0;
        #10
        clk=1; push=0; pop=1; din=0;
end
#10
$display("\n");

$display("pop while empty ");
clk=0; push=0; pop=1; din=0;
#10
clk=1; push=0; pop=1; din=0;
#10
$display("\n");

$display("simultaneuos push and pop (effectively only push because fifo is empty)");
clk=0; push=1; pop=1; din=8'd8;
#10
clk=1; push=1; pop=1; din=8'd8;
#10
$display("\n");

$display("B2B simultaneous push/pop (buffer behavior at rising edge)");
clk=0; push=1; pop=1; din=8'd88;
#10
clk=1; push=1; pop=1; din=8'd88;
#10
clk=0; push=1; pop=1; din=8'd41;
#10
clk=1; push=1; pop=1; din=8'd41;
#10
$display("\n");
```

```
$display("push some, then pop some");
repeat(6) begin
        #10
        clk=0; push=1; pop=0; din=din+8'd5;
        #10
        clk=1; push=1; pop=0; din=din;
end
#10
repeat(3) begin
        #10
        clk=0; push=0; pop=1; din=0;
        #10
        clk=1; push=0; pop=1; din=0;
end
#10

$display("\n");

$display("final reset");
clk=0; reset=1; push=0; pop=0; din=8'd99;
#10
clk=1; reset=1; push=0; pop=0; din=8'd99;
#10
$display("\n");

$display("End of FIFO Test");
$finish;

end

endmodule
```

5) Turn in the modules, testbench, and simulation evidence that it works.

```
[dthornburg@linux402604 HW4]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Nov  2 18:40 2025
Start of the FIFO Test


starting reset (x instead of 0 on dout to symbolize empty. 0 on dout doesnt necessarily mean nothing. I could be storing 0 in the fifo, but not x.)
push=0, pop=0, din=  0, dout=  x, full=x, empty=x, reset=1, clk=0, @                    0
push=0, pop=0, din=  0, dout=  X, full=0, empty=1, reset=1, clk=1, @                   10


push 10
push=1, pop=0, din= 10, dout=  X, full=0, empty=1, reset=0, clk=0, @                   20
push=1, pop=0, din= 10, dout= 10, full=0, empty=0, reset=0, clk=1, @                   30


push 20
push=1, pop=0, din= 20, dout= 10, full=0, empty=0, reset=0, clk=0, @                   40
push=1, pop=0, din= 20, dout= 10, full=0, empty=0, reset=0, clk=1, @                   50


push data until full
push=1, pop=0, din= 30, dout= 10, full=0, empty=0, reset=0, clk=0, @                   70
push=1, pop=0, din= 30, dout= 10, full=0, empty=0, reset=0, clk=1, @                   80
push=1, pop=0, din= 40, dout= 10, full=0, empty=0, reset=0, clk=0, @                   90
push=1, pop=0, din= 40, dout= 10, full=0, empty=0, reset=0, clk=1, @                  100
push=1, pop=0, din= 50, dout= 10, full=0, empty=0, reset=0, clk=0, @                  110
push=1, pop=0, din= 50, dout= 10, full=0, empty=0, reset=0, clk=1, @                  120
push=1, pop=0, din= 60, dout= 10, full=0, empty=0, reset=0, clk=0, @                  130
push=1, pop=0, din= 60, dout= 10, full=0, empty=0, reset=0, clk=1, @                  140
push=1, pop=0, din= 70, dout= 10, full=0, empty=0, reset=0, clk=0, @                  150
push=1, pop=0, din= 70, dout= 10, full=0, empty=0, reset=0, clk=1, @                  160
push=1, pop=0, din= 80, dout= 10, full=0, empty=0, reset=0, clk=0, @                  170
push=1, pop=0, din= 80, dout= 10, full=0, empty=0, reset=0, clk=1, @                  180
push=1, pop=0, din= 90, dout= 10, full=0, empty=0, reset=0, clk=0, @                  190
push=1, pop=0, din= 90, dout= 10, full=0, empty=0, reset=0, clk=1, @                  200
push=1, pop=0, din=100, dout= 10, full=0, empty=0, reset=0, clk=0, @                  210
push=1, pop=0, din=100, dout= 10, full=0, empty=0, reset=0, clk=1, @                  220
push=1, pop=0, din=110, dout= 10, full=0, empty=0, reset=0, clk=0, @                  230
push=1, pop=0, din=110, dout= 10, full=0, empty=0, reset=0, clk=1, @                  240
push=1, pop=0, din=120, dout= 10, full=0, empty=0, reset=0, clk=0, @                  250
push=1, pop=0, din=120, dout= 10, full=1, empty=0, reset=0, clk=1, @                  260
```

```
try pushing while full
push=1, pop=0, din=130, dout= 10, full=1, empty=0, reset=0, clk=0, @                  270
push=1, pop=0, din=130, dout= 10, full=1, empty=0, reset=0, clk=1, @                  280


pop first element
push=0, pop=1, din=  0, dout= 10, full=1, empty=0, reset=0, clk=0, @                  290
push=0, pop=1, din=  0, dout= 20, full=0, empty=0, reset=0, clk=1, @                  300


pop elements until empty
push=0, pop=1, din=  0, dout= 20, full=0, empty=0, reset=0, clk=0, @                  320
push=0, pop=1, din=  0, dout= 30, full=0, empty=0, reset=0, clk=1, @                  330
push=0, pop=1, din=  0, dout= 30, full=0, empty=0, reset=0, clk=0, @                  340
push=0, pop=1, din=  0, dout= 40, full=0, empty=0, reset=0, clk=1, @                  350
push=0, pop=1, din=  0, dout= 40, full=0, empty=0, reset=0, clk=0, @                  360
push=0, pop=1, din=  0, dout= 50, full=0, empty=0, reset=0, clk=1, @                  370
push=0, pop=1, din=  0, dout= 50, full=0, empty=0, reset=0, clk=0, @                  380
push=0, pop=1, din=  0, dout= 60, full=0, empty=0, reset=0, clk=1, @                  390
push=0, pop=1, din=  0, dout= 60, full=0, empty=0, reset=0, clk=0, @                  400
push=0, pop=1, din=  0, dout= 70, full=0, empty=0, reset=0, clk=1, @                  410
push=0, pop=1, din=  0, dout= 70, full=0, empty=0, reset=0, clk=0, @                  420
push=0, pop=1, din=  0, dout= 80, full=0, empty=0, reset=0, clk=1, @                  430
push=0, pop=1, din=  0, dout= 80, full=0, empty=0, reset=0, clk=0, @                  440
push=0, pop=1, din=  0, dout= 90, full=0, empty=0, reset=0, clk=1, @                  450
push=0, pop=1, din=  0, dout= 90, full=0, empty=0, reset=0, clk=0, @                  460
push=0, pop=1, din=  0, dout=100, full=0, empty=0, reset=0, clk=1, @                  470
push=0, pop=1, din=  0, dout=100, full=0, empty=0, reset=0, clk=0, @                  480
push=0, pop=1, din=  0, dout=110, full=0, empty=0, reset=0, clk=1, @                  490
push=0, pop=1, din=  0, dout=110, full=0, empty=0, reset=0, clk=0, @                  500
push=0, pop=1, din=  0, dout=120, full=0, empty=0, reset=0, clk=1, @                  510
push=0, pop=1, din=  0, dout=120, full=0, empty=0, reset=0, clk=0, @                  520
push=0, pop=1, din=  0, dout=  X, full=0, empty=1, reset=0, clk=1, @                  530


pop while empty
push=0, pop=1, din=  0, dout=  X, full=0, empty=1, reset=0, clk=0, @                  540
push=0, pop=1, din=  0, dout=  X, full=0, empty=1, reset=0, clk=1, @                  550
```

```
simultaneuos push and pop (effectively only push because fifo is empty)
push=1, pop=1, din=  8, dout=  X, full=0, empty=1, reset=0, clk=0, @                  560
push=1, pop=1, din=  8, dout=  8, full=0, empty=0, reset=0, clk=1, @                  570


B2B simultaneous push/pop (buffer behavior at rising edge)
push=1, pop=1, din= 88, dout=  8, full=0, empty=0, reset=0, clk=0, @                  580
push=1, pop=1, din= 88, dout= 88, full=0, empty=0, reset=0, clk=1, @                  590
push=1, pop=1, din= 41, dout= 88, full=0, empty=0, reset=0, clk=0, @                  600
push=1, pop=1, din= 41, dout= 41, full=0, empty=0, reset=0, clk=1, @                  610


push some, then pop some
push=1, pop=0, din= 46, dout= 41, full=0, empty=0, reset=0, clk=0, @                  630
push=1, pop=0, din= 46, dout= 41, full=0, empty=0, reset=0, clk=1, @                  640
push=1, pop=0, din= 51, dout= 41, full=0, empty=0, reset=0, clk=0, @                  650
push=1, pop=0, din= 51, dout= 41, full=0, empty=0, reset=0, clk=1, @                  660
push=1, pop=0, din= 56, dout= 41, full=0, empty=0, reset=0, clk=0, @                  670
push=1, pop=0, din= 56, dout= 41, full=0, empty=0, reset=0, clk=1, @                  680
push=1, pop=0, din= 61, dout= 41, full=0, empty=0, reset=0, clk=0, @                  690
push=1, pop=0, din= 61, dout= 41, full=0, empty=0, reset=0, clk=1, @                  700
push=1, pop=0, din= 66, dout= 41, full=0, empty=0, reset=0, clk=0, @                  710
push=1, pop=0, din= 66, dout= 41, full=0, empty=0, reset=0, clk=1, @                  720
push=1, pop=0, din= 71, dout= 41, full=0, empty=0, reset=0, clk=0, @                  730
push=1, pop=0, din= 71, dout= 41, full=0, empty=0, reset=0, clk=1, @                  740
push=0, pop=1, din=  0, dout= 41, full=0, empty=0, reset=0, clk=0, @                  760
push=0, pop=1, din=  0, dout= 46, full=0, empty=0, reset=0, clk=1, @                  770
push=0, pop=1, din=  0, dout= 46, full=0, empty=0, reset=0, clk=0, @                  780
push=0, pop=1, din=  0, dout= 51, full=0, empty=0, reset=0, clk=1, @                  790
push=0, pop=1, din=  0, dout= 51, full=0, empty=0, reset=0, clk=0, @                  800
push=0, pop=1, din=  0, dout= 56, full=0, empty=0, reset=0, clk=1, @                  810


final reset
push=0, pop=0, din= 99, dout= 56, full=0, empty=0, reset=1, clk=0, @                  820
push=0, pop=0, din= 99, dout=  X, full=0, empty=1, reset=1, clk=1, @                  830


End of FIFO Test
$finish called from file "tb.v", line 122.
$finish at simulation time                 840
           V C S   S i m u l a t i o n   R e p o r t
Time: 840
CPU Time:      0.190 seconds;       Data structure size:   0.0Mb
Sun Nov  2 18:40:31 2025
[dthornburg@linux402604 HW4]$
```